Practical:03

Prac:3.1: Understanding Activity Life Cycle

Problem Statement: Develop an Android application to demonstrate the Activity Life Cycle by observing life cycle method calls during different user interactions.

Procedure:

Language: Java
Template: Empty Views Activity

STEP 1: Create a New Android Project

Open Android Studio

- Launch Android Studio
- Click New Project

Select Project Template

- Choose Empty Views Activity
- Click Next

Project Configuration (VERY IMPORTANT)

Fill the details exactly like this (you may change the name slightly if needed):

- Name:
  ActivityLifeCycleDemo
- Package name:
  com.example.activitylifecycle
- Save location:
  (Keep default)
- Language:
  Java
- Minimum SDK:
  API 21: Android 5.0 (Lollipop)
- Build configuration language:
  Groovy (default)

Click Finish

STEP 2: Designing the UI (TextView for Life Cycle messages)

Replace the Entire XML Code

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"

    android:layout_height="match_parent"
```

```
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">
    <TextView
        android:id="@+id/tvLifecycle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Activity Life Cycle Status"
        android:textSize="20sp"
        android:textStyle="bold" />
</LinearLayout>
```

Why This UI? (Understand Concept)

- LinearLayout → arranges items vertically
- TextView → shows which life cycle method is currently running
- tvLifecycle → will be updated from Java code

MainActivity.java

SourceCode

```
package com.example.activitylifecycledemo;
import android.os.Bundle;
import android.util.Log;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    TextView tvLifecycle;
    String TAG = "ActivityLifeCycle";
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        tvLifecycle = findViewById(R.id.tvLifecycle);
        tvLifecycle.setText("onCreate() called");

        Log.d(TAG, "onCreate() executed");
    }
    @Override
    protected void onStart() {
        super.onStart();
        tvLifecycle.setText("onStart() called");
        Log.d(TAG, "onStart() executed");
    }
```

```
    @Override
    protected void onResume() {
        super.onResume();
        tvLifecycle.setText("onResume() called");
        Log.d(TAG, "onResume() executed");
    }
    @Override
    protected void onPause() {
        super.onPause();
        Log.d(TAG, "onPause() executed");
    }
    @Override
    protected void onStop() {
        super.onStop();
        Log.d(TAG, "onStop() executed");
    }
    @Override
    protected void onDestroy() {
        super.onDestroy();
        Log.d(TAG, "onDestroy() executed");
    }
}
```

Understand What We Did

- onCreate() → app starts
- onStart() → activity becomes visible
- onResume() → user interacts
- onPause() → app goes to background
- onStop() → app not visible
- onDestroy() → activity closed

We used:

- TextView → to show state on screen
- Log.d() → to track life cycle in Logcat

STEP 4: Run the App and Observe Activity Life Cycle

Run the Application

- Click the ▶ Run button in Android Studio
- Select:

Observe Output on Mobile Screen

When the app opens, you will see:

onResume() called

(You may briefly see onCreate → onStart → onResume internally.)

Open Logcat

- Bottom panel → Logcat
- Select:

  Device: your emulator/phone

  App: your application
- In search box, type:

ActivityLifeCycle

Perform User Actions & Observe Life Cycle

User Action	Life Cycle Methods Called
Launch app	onCreate → onStart → onResume
Press Home	onPause → onStop
Return to app	onRestart → onStart → onResume
Press Back	onPause → onStop → onDestroy

(Logcat will clearly show these messages.)

What to Capture for Output

For practical submission, take:

- Screenshot of app screen
- Screenshot of Logcat showing life cycle logs

Practical–3.2: Exploring Multiple Activities

Aim

To develop an Android application that demonstrates interaction between multiple activities by navigating between screens and presenting user-specific content dynamically.

Problem Statement

Design an Android application consisting of two activities where the first activity acts as a welcome screen to collect basic user input, and the second activity displays a personalized message based on the input received, demonstrating activity-to-activity communication.

STEP 1: Create a New Android Project

Open Android Studio

- Click New Project

Select Template

- Choose Empty Views Activity
- Click Next

Project Configuration

- Name:
  WelcomeMessageApp
- Language:
  Java
- Minimum SDK:
  API 21 (Android 5.0)
- Build configuration language:
  Groovy (default)

Click Finish

Wait for Gradle sync to complete.

activity_mainXML: SOURCECODE:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome!"
        android:textSize="26sp"
        android:textStyle="bold"
        android:layout_marginBottom="20dp" />
```

```
    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter your name"
        android:inputType="textPersonName"
        android:layout_marginBottom="16dp" />
    <Button
        android:id="@+id/btnNext"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Continue" />
</LinearLayout>
```

Concept & Logic

- TextView → welcomes the user
- EditText → takes user input (name)
- Button → moves to second activity

STEP 3: Create Second Activity (Display Screen)

Create New Activity

- Right-click on java → com.example.multipleactivities
- Select New → Activity → Empty Views Activity

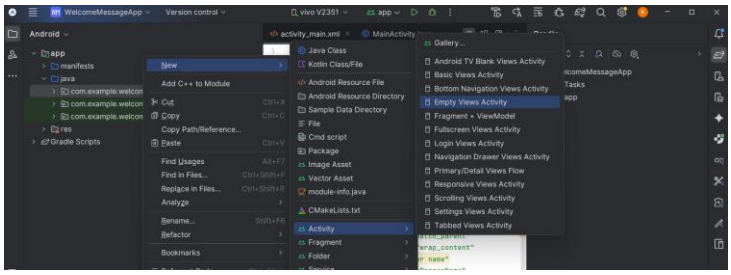
Activity Configuration

Fill details exactly:

- Activity Name:
  SecondActivity
- Layout Name:
  activity_second
- Language:
  Java

Click Finish

Wait for Gradle to sync.



Activity_second.xml:code

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">
    <TextView
        android:id="@+id/tvMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello User!"
        android:textSize="24sp"
        android:textStyle="bold" />
</LinearLayout>
```

Second_activity.java

```
package com.example.welcomemessageapp;
import android.os.Bundle;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
public class SecondActivity extends AppCompatActivity {
    TextView tvMessage;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);
        // Link TextView
        tvMessage = findViewById(R.id.tvMessage);
        // Get data sent from MainActivity
        String name = getIntent().getStringExtra("username");
        // Display personalized message
        tvMessage.setText("Hello, " + name + "! Welcome to the App.");
    }
}
```

MainActivity.java

```
package com.example.welcomemessageapp;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    EditText etName;
    Button btnNext;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        // Linking UI components with Java
        etName = findViewById(R.id.etName);
        btnNext = findViewById(R.id.btnNext);
        // Button click event
        btnNext.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                // Get user input
                String name = etName.getText().toString();
                // Create Intent to move to SecondActivity
                Intent intent = new Intent(MainActivity.this,
com.example.welcomemessageapp.SecondActivity.class);
                // Pass data using Intent
                intent.putExtra("username", name);
                // Start second activity
                startActivity(intent);
            }
        });
    }
}
```

Run the application

Practical:3.3

Aim

To create an interactive Android application that uses fragments to divide the screen into multiple sections, allowing users to input data in one fragment and display it in another fragment dynamically.

Problem Statement

Develop an Android application containing two fragments inside one activity:

- Input Fragment: Accepts user input (e.g., name, message)
- Display Fragment: Shows the entered input in real-time when a button is clicked.
  This demonstrates fragment interaction, dynamic UI updates, and fragment lifecycle understanding.

Open Android Studio → New Project

Select Empty Views Activity

Project Configuration:

Name: FragmentCommunicationApp

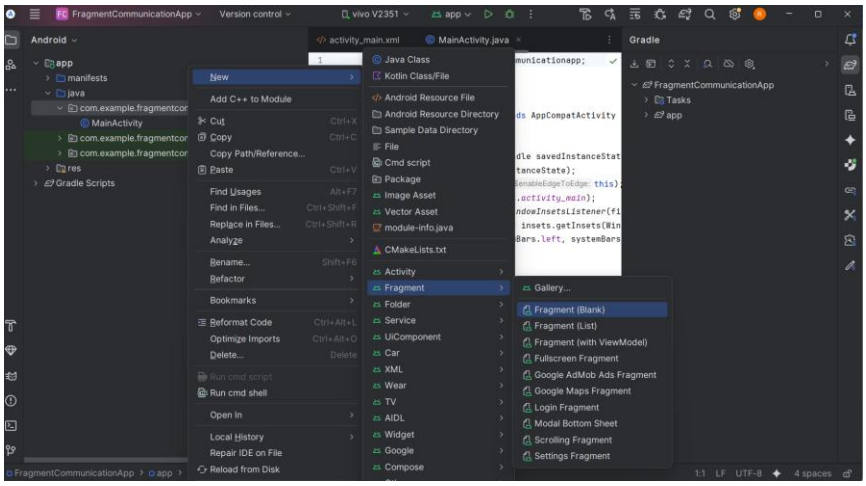
Package: com.example.fragmentsapp

Language: Java

Minimum SDK: API 21

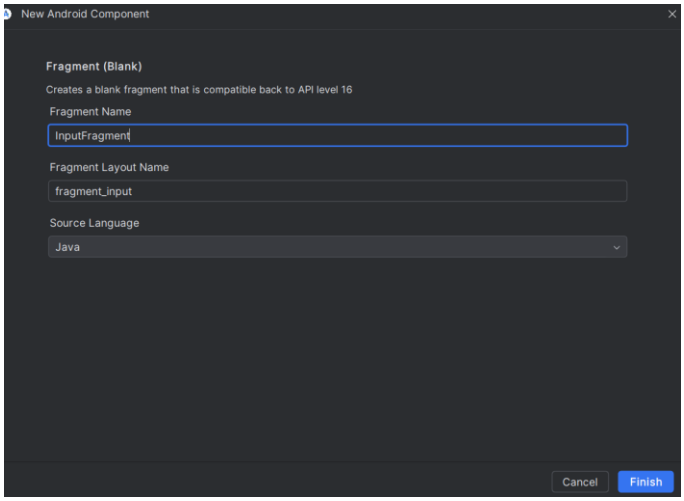
Finish and wait for Gradle sync.

STEP 2: Create Two Fragments



Fragment A (Input Fragment)

- Right-click java → New → Fragment → Fragment (Blank)
- Name: InputFragment
- Layout: fragment_input.xml
- Finish



Fragment B (Display Fragment)

- Right-click java → New → Fragment → Fragment (Blank)
- Name: DisplayFragment
- Layout: fragment_display.xml
- Finish

STEP 3: Design Fragment Layouts

3A: fragment_input.xml (User Input)

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp"
    android:gravity="center">
    <EditText
        android:id="@+id/etMessage"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter your message" />
```

```
    <Button
        android:id="@+id/btnSend"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Send"
        android:layout_marginTop="16dp" />
</LinearLayout>
```

3B: fragment_display.xml (Display Output)

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:padding="24dp">
    <TextView
        android:id="@+id/tvDisplay"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Your message will appear here"
        android:textSize="20sp"
        android:textStyle="bold" />
</LinearLayout>
```

STEP 4: Add Fragments to MainActivity Layout

Open activity_main.xml and replace with:

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <FrameLayout
        android:id="@+id/fragment_container_input"
        android:layout_width="0dp"
        android:layout_weight="1"
        android:layout_height="match_parent"/>
```

```
    <FrameLayout
        android:id="@+id/fragment_container_display"
        android:layout_width="0dp"
        android:layout_weight="1"
        android:layout_height="match_parent"/>
</LinearLayout>
```

Logic:

- Two FrameLayouts divide the screen into left (input) and right (display).
- Fragments will be loaded dynamically into these containers.

STEP 5: Write Java Code for Fragments

5A: InputFragment.java

```
package com.example.fragmentcommunicationapp;
import android.content.Context;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.EditText;
import androidx.fragment.app.Fragment;
public class InputFragment extends Fragment {
    EditText etMessage;
    Button btnSend;
    OnMessageSendListener listener;
    public interface OnMessageSendListener {
        void onMessageSend(String message);
    }
    @Override
    public void onAttach(Context context) {
        super.onAttach(context);
        if(context instanceof OnMessageSendListener){
            listener = (OnMessageSendListener) context;
        } else {
            throw new RuntimeException(context.toString() + " must implement
OnMessageSendListener");
        }
    }

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {

        View view = inflater.inflate(R.layout.fragment_input, container, false);
        etMessage = view.findViewById(R.id.etMessage);
```

```
        btnSend = view.findViewById(R.id.btnSend);

        btnSend.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String message = etMessage.getText().toString();
                listener.onMessageSend(message);
            }
        });

        return view;
    }
}
```

5B: DisplayFragment.java

```
package com.example.fragmentcommunicationapp;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;
import androidx.fragment.app.Fragment;
public class DisplayFragment extends Fragment {
    TextView tvDisplay;
    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_display, container, false);
        tvDisplay = view.findViewById(R.id.tvDisplay);
        return view;
    }
    public void updateMessage(String message){
        tvDisplay.setText(message);
    }
}
```

STEP 6: MainActivity.java (Handles Fragment Communication)

```
package com.example.fragmentcommunicationapp;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.FragmentManager;
import androidx.fragment.app.FragmentTransaction;
public class MainActivity extends AppCompatActivity implements
InputFragment.OnMessageSendListener {
    DisplayFragment displayFragment;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        InputFragment inputFragment = new InputFragment();
```

```
        displayFragment = new DisplayFragment();
        FragmentManager fm = getSupportFragmentManager();
        FragmentTransaction ft = fm.beginTransaction();
        ft.replace(R.id.fragment_container_input, inputFragment);
        ft.replace(R.id.fragment_container_display, displayFragment);
        ft.commit();
    }
    @Override
    public void onMessageSend(String message) {
        displayFragment.updateMessage(message);
    }
}
```

run